

Data Formats and Validations



SoftUni Team
Technical Trainers



SoftUni



Software University

<https://about.softuni.bg>

1. Data Formats

- Types of Formats, Comparison

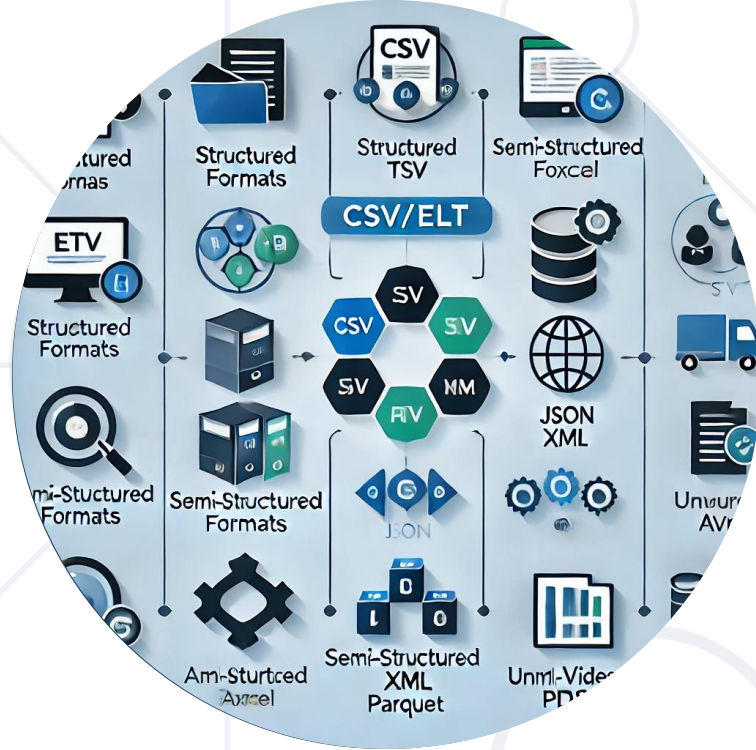
2. Data Sources

- Flat Files, APIs

3. Data Validation Techniques with Pandera

- Introduction to Pandera
- Pandera Schemas Basics
- Advanced Validation





Data Formats

Types of Formats and Comparison

- **Structured** Data – CSV, EXCEL – Rows and Columns, easy to query
 - Common Tools – Pandas, SQL
- **Semi-Structured** Data – JSON, XML, Parquet – Flexible schema, hierarchical
 - Common Tools – Pandas, PySpark, APIs
- **Unstructured** Data – Images, Text, Videos
 - Tools – TensorFlow, OpenCV

Type of Formats	Characteristics	When to Use	Tools/Technologies
Structured Data	- Organized in Rows and Columns - Easy queried with SQL - Fixed schema	- Analytical tasks requiring structured, tabular data - Reporting and dashboards - Loading data in relation data base	Pandas, SQL, EXCEL, Tableau
Semi-Structured Data	- Flexible schema - Supports hierarchical structures - More compact formats like Parquet for big data	- Storing or exchanging data between application (e.g., APIs) - Use in big data platforms (e.g., Hadoop, Spark) - Data lakes and flexible analytics	Pandas, PySpark, Snowflake, APIs
Unstructured Data	- No fixed format - Includes media files (images, videos), text, etc. - Requires specialized processing tools	- Machine learning or deep learning tasks - Natural language processing (NLP) - Image or video analysis	TensorFlow, OpenCV, NLP libraries

- Flat Files
 - **CSV, JSON, Parquet** files stored **locally** or **cloud storage**
 - Tools – **Pandas, AWS SDKs** (boto3 or S3Hook, for s3)
- APIs
 - **Rest APIs** and their typical response formats (**JSON, XML**)
 - Tools – requests library for **GET** and **POST requests**

- **What is Pandera?**
 - A **Python library** for validating **Pandas DataFrames**
 - Ensures **data integrity** by defining schemas with constraints
- **Why use Pandera?**
 - **Automates validation**, ensuring data is clean before transformations
 - **Integrates seamlessly** into ETL Pipelines

- **Pandera Schema Basics** – Define column types, null constraints, and custom checks

```
import pandas as pd
import pandera as pa
schema = pa.DataFrameSchema({
    employe_id = pa.Column(int),
    first_name = pa.Column(str, required=False, nullable=True)})
```



```
data = pd.DataFrame({
    employe_id = [1, 2, 3],
    first_name = ["Alice", "Bob", "Charlie"]
})
try:
    validated_data = schema.validate(data)
    print(f"Validation successful, {validated_data}")
except pa.errors.SchemaError as e:
    print(f"Validation failed {e}")
```

- **Custom Column Checks**
- **Multi Column Checks**
- **Error Reporting** – Generate detailed error response



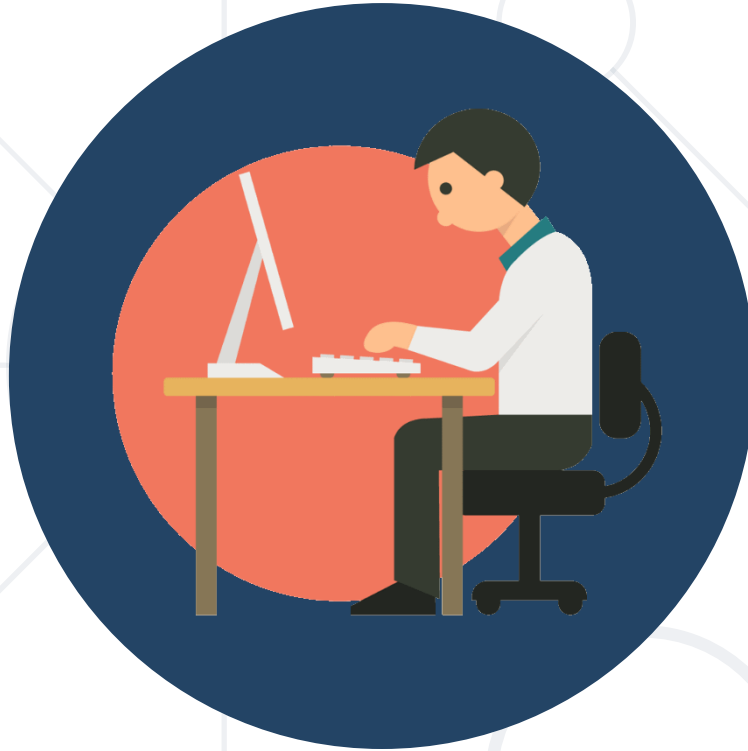
Advanced Validations

```
pa.CONFIG.show_failure_tracebacks = True
schema = pa.DataFrameSchema( { "employee_id": pa.Column(int, checks=pa.Check(lambda x: x > 0, error="Employee ID
must be positive")),
"hire_date": pa.Column(pa.DateTime),
"salary": pa.Column(float, checks=pa.Check(lambda x: x >= 30000.0, error="Salary must be at least $30,000")),
"bonus": pa.Column(float),
"total_compensation": pa.Column(float), },

checks=pa.Check( lambda df: df["total_compensation"] == df["salary"] + df["bonus"], error="Total compensation must
equal salary + bonus" ) )

data = pd.DataFrame({ "employee_id": [1, -1, 3], "hire_date": ["2023-01-01", "2023-02-01", "2023-03-01"],
"salary": [50000.0, 20000.0, 70000.0], "bonus": [5000.0, 1000.0, 3000.0], "total_compensation": [55000.0, 30000.0,
75000.0]})

try:
    validated_data = schema.validate(data)
    print("Validation successful!")
except pa.errors.SchemaError as e:
    print("Validation failed! Detailed error report:")
    print(e.failure_cases)
```



Practice

Exercise: Data Extraction - JSON, S3, SQL, APIs

- The **types** and **characteristics** of data formats
- How to **validate data effectively** using **Pandera** in **ETL pipelines**
- **Advanced validation techniques** to ensure data quality

